4th Forced-Logout Incident â€” 2026-08-19 00:37:11 UTC+2

Revision: 1 **Last modified:** 2026-08-19T01:15:00Z **Tracker:** BOB-123 (to be minted); cross-refs BOB-116 (2nd), BOB-120 (3rd) **Verdict (Â§11.4.6):** ROOT-CAUSE MECHANISM NARROWED; DIRECT SIGKILL INITIATOR STILL UNATTRIBUTABLE

What happened

At `2026-08-19 00:37:11 CEST` `user@1000.service` was SIGKILLed. Every child process (gsd-*, gnome-shell, Xwayland, mutter, wireplumber, ibus-*, tmux client, gvfs*, drweb-gui-agent) received SIGKILL. GDM immediately respawned a greeter on tty1. Operator perceived a full logout. Session resumed at `00:50:09 CEST` (13m dead).

Three subagents were mid-flight when the kill hit:

- Task 86 subagent (BOB-108 sync db-to-md refix) â€” commit `78c2f66` **landed before kill**
- Task IPTorrents subagent (BOB-122 seed/leech parser) â€” commit `26e107b` **landed before kill**
- Task 85 subagent (BOB-121 containers audit) â€” commit `4ff3355` **landed before kill**
- Task BOB-118 badge computer â€” **partial work in tree, killed mid-authoring**

Rescue commit `d21ff34` (or successor) recovers the BOB-118 partial work after relogin.

Breakthrough: PAM session_close synchronicity

Direct grep across all 3 most-recent forced-logout events (2026-08-18 20:50:59, 2026-08-18 23:45:49, 2026-08-19 00:37:11) reveals an IDENTICAL signature at each kill timestamp:

```
Aug 19 00:37:11 gdm-password][3115126]: pam_tcb(gdm-password:session):
    Session closed for milosvasic
Aug 19 00:37:11 audit[3115126]: op=PAM:session_close
    grantors=pam_tcb,pam_mktemp,pam_limits,pam_loginuid,pam_systemd,pam_na
    acct="milosvasic" exe="/usr/libexec/gdm-session-worker"
    hostname=nezha addr=? terminal=/dev/tty2 res=success
Aug 19 00:37:11 systemd[1]: user@1000.service: Main process exited,
    code=killed, status=9/KILL
Aug 19 00:37:11 systemd[1]: user@1000.service: Killing process ...
```

Every SIGKILL of user@1000 in the last 3 incidents is preceded (or coincident within one journal timestamp) by a GDM PAM session_close for milosvasic on tty2. This is not coincidence â€” it is the mechanism.

Contradiction: Linger=YES

```
$ loginctl show-user milosvasic | grep -iE "Linger|State|Sessions"
State=active
Sessions=33 32
Linger=yes
```

With `Linger=yes`, `user@1000.service` **should** survive session teardown by design â€” that is exactly what linger means. Yet the SIGKILL fires anyway. Either:

1. systemd on this host has a bug in the linger-preservation path when GDM tears down a session, OR
2. A component ABOVE systemd (an out-of-band `loginctl terminate-user` or `systemctl stop user@1000.service` invocation) is issuing the stop request AFTER the PAM close, and the SIGKILL is systemd honoring that stop request, OR
3. A pattern in the SDD-orchestration machinery or in one of the sibling containers is invoking a stop path the journal does not attribute.

Journal search for `loginctl.*terminate|systemctl.*stop.*user@1000|kill.*user@1000` across the 5-minute window returns EMPTY â€” the initiator is not self-attributed.

What ruled OUT (all 4 incidents)

- Kernel OOM-killer: NEVER fired (`journalctl -k | grep oom-kill` empty)
- systemd-oomd: NEVER triggered (Avg60 PSI never crossed 90%)
- CONST-033 forbidden power mechanism (host uptime intact + no `suspend/hibernate` in kernel journal)
- IdleAction: `IdleAction=ignore` (verified)
- `KillUserProcesses`: unset in `/etc/systemd/logind.conf`. The systemd default is distro-patched (some ship yes, some no); the authoritative check is `busctl get-property org.freedesktop.login1 /org/freedesktop/login1 org.freedesktop.login1.Manager KillUserProcesses` at runtime (Â§11.4.6 â€” donâ€™t assume, check). Regardless, `Linger=yes` should override this behavior for user@1000.service itself.

Preventive gates that DIDNâ€™T fire (Â§11.4.6)

- `boba-resource-pressure-check.timer` (task #77 landed 2026-08-18) fires hourly INSIDE `user@1000.service` â€” so it dies WITH

user@1000, cannot observe a kill. Architectural gap acknowledged in BOB-120 doc.
- No in-scope monitor can observe the SIGKILL of the scope it runs in.
- The only path to catch the SIGKILL source is an OUT-OF-SCOPE monitor â€” system.slice audit rule / systemd-journal-audit â€” which requires operator sudo.

Recommended next step (needs operator authorization)

Install kernel audit watch on `loginctl` and `systemctl` binaries + a signal-delivery watch. This will attribute the NEXT SIGKILL to a specific initiator (PID, UID, cmdline) with cryptographic certainty:

```
# ROOT-OWNED persistent audit rules
sudo auditctl -w /usr/bin/loginctl -p x -k logout_investigation
sudo auditctl -w /usr/bin/systemctl -p x -k logout_investigation
sudo auditctl -a always,exit -F arch=b64 -S kill -F a1=9 -k sigkill_invest
```

This is the SHORTEST path to a real fix. Without it, all 4 incidents remain Â§11.4.6 `UNCONFIRMED` at the â€œdirect SIGKILL initiatorâ€ layer, and any â€œfixâ€ I propose is a guess.

Unilateral defensive move (implemented now, no consent needed)

Reducing parallel-subagent default to 1 for the remainder of this session. Reasoning: all 4 incidents occurred during heavy parallel subagent load (3+ concurrent claude-code + MCP fleet). Whether load is CAUSAL or merely CORRELATED is unknown (Â§11.4.6), but reducing load narrows the reproduction surface and is reversible on operator ask.

Full journalctl evidence

See `docs/qa/BOB-123/incident-4-forensics.log` â€” captured window `2026-08-19 00:35:00` â†’ `00:38:00`.

Cross-references

- `docs/incidents/2026-08-18-perceived-forced-logout-2nd.md` (BOB-116)
- `docs/incidents/2026-08-18-3rd-forced-logout.md` (BOB-120)
- `~/.claude-claude4/projects/.../memory/forced_logout_incidents.md` playbook
- BOB-116 / BOB-120 / BOB-123 all Â§11.4.6 UNCONFIRMED at initiator layer

---

RETROSPECTIVE Â§11.4.6 CORRECTION â€” RESOLVED via BOB-126 (added 2026-08-19T17:32:00Z)

This docâ€™s original hypothesis (PAM/Linger contradiction and/or various downstream mechanisms) was **WRONG**. The 7th forced-logout incident (BOB-126, 2026-08-19 16:43:43) captured the REAL root cause via kernel audit trail:

```
audit[399861]: SYSCALL syscall=62 a0=ffffffff a1=9
  pid=399861 auid=1000 comm="pytest" exe="/usr/bin/python3.14"
  key="sigkill_investigation"
```

A `pytest` process called `kill(-1, SIGKILL)`. Root cause: `tests/unit/merge_service/test_deadline_tunable.py::test_deadline_hit_flag_true_when_readline_times_` created `AsyncMock()` without setting `mock.pid` as int. Production `_search_public_tracker` called `os.killpg(os.getpgid(proc.pid), SIGKILL)`. `MagicMock.__int__` defaults to 1, so `os.getpgid(1) == 1` â†’ `os.killpg(1, SIGKILL)` â†’ glibc â†’ `kill(-1, SIGKILL)` = SIGKILL every UID-1000 process. Bug existed since 2026-04-24 (~4 months).

PAM session_close was a DOWNSTREAM effect, not the initiator. The `Linger=yes` protection was irrelevant â€” the SIGKILL came from userspace (a UID-1000 process signaling its own UID-group), not from systemdâ€™s session lifecycle.

Fix chain:

- `ad4b46a` â€” boba: `search.py` int-guard + test hardening + Â§11.4.115 RED regression guard
- `502586c` â€” constitution: NEW universal Â§11.4.263 anchor covering Python/Go/Rust/Bash/C
- `bf01cf3` â€” boba: constitution pointer bump
- `e984f4b` â€” boba: workable_items.db chain-close + regenerated docs

Verification: 863/863 unit tests PASS + 14/14 Go race packages clean + 50+ min elapsed vs prior ~37-38 min cadence with no 8th incident.

Â§11.4.238 escape audit logged in `docs/QA_DISCOVERY_LEDGER.md` Rev 12.

See `docs/incidents/2026-08-19-6th-forced-logout.md` (Rev 4+) for the full attribution.